

Version 3.0 — Édition IA & Machine Learning

PitSim-F1

Simulateur Stratégique de Course de Formule 1
sous Contraintes Météorologiques et Intelligence Artificielle

Projet	Statut	Date
PitSim-F1	En Développement	20 mai 2026
Framework	Version	Domaine
Django 4.2 / Python 3.11	v3.0	IA & Data Science

Optimisation Combinatoire | Machine Learning | Programmation Dynamique |
Clustering

Table des matières

1	Présentation Générale du Projet	2
1.1	Contexte : La Stratégie en Formule 1	2
1.2	Vision du Produit	2
1.3	Évolution par rapport à la Version 2.0	2
2	Problématique Technique	2
2.1	Énoncé du Problème	2
2.2	Modèle Mathématique Complet	3
2.2.1	Équation d'évolution du temps au tour (v3.0)	3
2.2.2	Fonction objectif	3
2.2.3	Récurrence de programmation dynamique	3
2.2.4	Pénalité météorologique	3
3	Modules Fonctionnels	4
3.1	F1 — Module de Configuration de Course	4
3.2	F2 — Moteur de Simulation (avec ML)	4
3.3	F3 — Moteur d'Optimisation par Programmation Dynamique	4
3.4	F4 — Dashboard de Restitution Visuelle	5
3.5	IA-1 — Modèle de Prédiction de Dégradation (ML Supervisé)	5
3.5.1	Données d'entraînement	5
3.5.2	Pipeline d'entraînement	6
3.6	IA-2 — Clustering de Circuits (Apprentissage Non Supervisé)	6
3.6.1	Features de clustering	6
3.6.2	Pipeline	6
4	Données Métier	7
4.1	Matrice des Écuries	7
4.2	Tableau des Pneumatiques	7
5	Architecture Technique	7
5.1	Stack Technologique	7
5.2	Architecture MVT Django	8
5.3	Organisation du Dépôt	9
6	Exigences Non-Fonctionnelles	9
7	Planning et Livrables	10
7.1	Planning Prévisionnel — 14 Semaines	10
7.2	Livrables Attendus	10
7.3	Critères d'Acceptation	10
8	Glossaire	11

1. Présentation Générale du Projet

1.1 Contexte : La Stratégie en Formule 1

En Formule 1 moderne, la performance brute d'une monoplace ne garantit plus la victoire. Les courses se gagnent sur le muret des stands, à travers une gestion stratégique millimétrée des relais (*stints*). Chaque décision — quand s'arrêter, avec quel pneumatique repartir — résulte d'un calcul en temps réel intégrant des dizaines de variables : usure des gomme, température de piste, météo, sous-cuts adverses, et gestion du trafic.

Les écuries de pointe (Red Bull, McLaren, Ferrari) investissent massivement dans des outils de simulation propriétaires. **PitSim-F1 v3.0** ambitionne de reproduire fidèlement ce niveau de sophistication en y intégrant trois couches d'intelligence artificielle absentes de la version précédente.

1.2 Vision du Produit

PitSim-F1 v3.0 est une application web Django qui permet à un ingénieur de course virtuel de :

- configurer un scénario de course complet (écurie, circuit, météo) ;
- simuler l'évolution des temps au tour via un modèle de dégradation entraîné sur données réelles (ML supervisé) ;
- obtenir une stratégie optimale calculée par programmation dynamique ($O(K \cdot N^2)$, 1 à 3 arrêts) avec détection du point de croisement ;
- bénéficier d'une classification automatique des circuits par profil thermique (clustering non supervisé) ;
- visualiser les résultats sur un dashboard interactif avec export.

1.3 Évolution par rapport à la Version 2.0

2. Problématique Technique

2.1 Énoncé du Problème

Problématique centrale

« Comment minimiser le temps total d'une course de Formule 1 en modélisant l'impact croisé de la dégradation des pneumatiques — prédite par apprentissage automatique —, des profils aérodynamiques des écuries actuelles, et des transitions météorologiques imprévisibles (sec $\leftrightarrow$ pluie), tout en déterminant par programmation dynamique la politique de pit-stop optimale (tours d'arrêt et choix de gomme) ? »

Composant	v2.0 (baseline)	v3.0 (cible)
Modèle de dégradation	Équation fixe $\alpha \cdot \gamma \cdot n^\delta$	Random Forest / XGBoost entraîné sur données réelles
Algorithme d'optimisation	Programmation dynamique $O(KN^2)$ — équation fixe	Programmation dynamique $O(KN^2)$ — dégradation ML intégrée
Classification des circuits	Manuelle (table statique)	K-Means sur données historiques
Météo	Événement injecté manuellement	Scénario probabiliste issu du clustering

TABLE 1 – Comparatif fonctionnel v2.0 vs v3.0

2.2 Modèle Mathématique Complet

2.2.1 Équation d'évolution du temps au tour (v3.0)

Dans la version 3.0, le terme de dégradation $\alpha_{\text{gomme}} \cdot \gamma_{\text{écurie}} \cdot n^\delta$ est remplacé par la sortie du modèle ML :

$$t(n) = t_{\text{base}} - \beta_{\text{essence}} \cdot n_{\text{total}} + \underbrace{\hat{f}_{\text{ML}}(n, T_{\text{piste}}, s_{\text{conduite}}, c_{\text{circuit}})}_{\text{dégradation prédite}} + \Delta t_{\text{météo}}(n) \quad (1)$$

où $\hat{f}_{\text{ML}}$ est le modèle supervisé (Random Forest ou XGBoost) entraîné sur les données historiques de courses F1.

2.2.2 Fonction objectif

$$T_{\text{total}} = \sum_{n=1}^N t(n) + \sum_{k=1}^K (T_{\text{pitlane}} + T_{\text{immob,écurie}}) \quad (2)$$

2.2.3 Récurrence de programmation dynamique

Soit $T[k][n]$ le coût minimal d'une course à k arrêts dont le dernier a lieu au tour n :

$$T[k][n] = \min_{m < n} \left(T[k-1][m] + T_{\text{pit}} + \sum_{i=m+1}^n t(i) \right) \quad (3)$$

La stratégie optimale globale est $\min_{k \in \{1,2,3\}, n} T[k][n]$, de complexité $O(K \cdot N^2)$.

2.2.4 Pénalité météorologique

$$\Delta t_{\text{météo}}(n) = \begin{cases} 0 & \text{si gomme adaptée aux conditions} \\ \lambda_{\text{inadapté}} \cdot (1 + \mu \cdot n_{\text{stint}}) & \text{si gomme inadaptée} \end{cases} \quad (4)$$

avec $\lambda_{\text{inadapté}} \in [3, 0; 8, 0]$ s et μ le facteur d'aggravation progressive du mauvais grip.

3. Modules Fonctionnels

L'application est organisée en **cinq modules** : les quatre modules historiques (F1–F4) complétés par deux nouveaux modules IA (IA-1, IA-2).

3.1 F1 — Module de Configuration de Course

Rôle : Point d'entrée de l'application. L'utilisateur configure le scénario complet avant simulation.

Exigences F1

- Sélection d'écurie avec chargement dynamique des coefficients ORM (γ , T_{pit} , δ).
- Choix du circuit via liste issue du clustering IA-3 (profil thermique automatiquement associé).
- Définition de la météo initiale : Sec / Humide / Détrempé.
- Injection d'un événement pluvieux à un tour spécifique.
- Sélection du composé de départ parmi 5 types : Soft, Medium, Hard, Inter, Wet.
- Paramétrage du niveau d'agressivité de conduite (faible / moyen / élevé) — nouvelle entrée du modèle ML.

3.2 F2 — Moteur de Simulation (avec ML)

Rôle : Calcul tour par tour des temps de passage. La dégradation n'est plus calculée par une équation fixe mais prédite par le modèle supervisé (IA-1).

Exigences F2

- Appel au modèle ML sérialisé (fichier `.pkl`) pour obtenir $\hat{f}_{\text{ML}}(n, T_{\text{piste}}, s, c)$ à chaque tour.
- Modélisation des 5 types de gommes avec leurs plages d'utilisation optimales.
- Application des pénalités météo selon l'équation (4).
- Prise en compte de l'allègement carburant (β_{essence} par tour).
- Fallback sur l'équation analytique si le modèle ML n'est pas disponible (mode dégradé).

3.3 F3 — Moteur d'Optimisation par Programmation Dynamique

Rôle : Déterminer la stratégie de pit-stop optimale (tours d'arrêt et composés) en explorant toutes les combinaisons viables à 1, 2 ou 3 arrêts via la récurrence (3).

Exigences F3

- Résolution par programmation dynamique : complexité $O(K \cdot N^2)$.
- Exploration exhaustive des stratégies à 1, 2 et 3 arrêts (règle sportive : $K_{\max} = 3$).
- Construction de la matrice des coûts $T[k][n]$ à partir des temps simulés par le moteur F2 (dégradation ML).
- Détection automatique du point de croisement (*crossover*) : tour auquel changer de gomme devient plus rentable que continuer.
- Prise en compte des pénalités météo dans le calcul des coûts de stint.
- Retour d'un objet JSON structuré : tours d'arrêt, gommes utilisées, T_{total} par stratégie, stratégie optimale identifiée.
- Temps de calcul ≤ 500 ms pour $N = 70$ tours.

3.4 F4 — Dashboard de Restitution Visuelle

Rôle : Visualisation des résultats de simulation et comparaison des stratégies.

Exigences F4

- Affichage du verdict optimal avec détail des relais (tours, gommes, temps).
- Graphique interactif Chart.js : évolution du temps au tour par stratégie.
- Comparaison tabulaire : 1 arrêt vs 2 arrêts vs 3 arrêts vs stratégie optimale.
- Indicateur de confiance du modèle ML (R^2 , intervalle de prédiction).
- Export CSV et PNG du graphique.

3.5 IA-1 — Modèle de Prédiction de Dégradation (ML Supervisé)

Objectif : Remplacer l'équation analytique fixe par un modèle entraîné sur des données réelles de courses F1, afin d'obtenir une dégradation contextuelle et réaliste.

3.5.1 Données d'entraînement

Feature	Type	Description
n	Entier	Tour depuis le dernier arrêt
T_{piste}	Flottant (°C)	Température de la piste
s_{conduite}	Catégoriel	Agressivité (faible / moyen / élevé)
c_{circuit}	Catégoriel	Identifiant du circuit (cluster)
type_gomme	Catégoriel	Soft / Medium / Hard / Inter / Wet
$\gamma_{\text{écurie}}$	Flottant	Facteur multiplicateur de l'écurie
Cible	Flottant (s)	Perte de temps due à la dégradation

TABLE 2 – Features et cible du modèle ML supervisé (IA-1)

3.5.2 Pipeline d'entraînement

1. Collecte et nettoyage des données historiques de télémétrie F1 (sources ouvertes : FastF1, Ergast API).
2. Ingénierie des features : encodage One-Hot des variables catégorielles, normalisation StandardScaler.
3. Entraînement comparatif : Random Forest, XGBoost, Gradient Boosting.
4. Validation croisée 5-fold ; métrique principale : RMSE sur la perte de temps au tour.
5. Sérialisation du meilleur modèle au format `joblib` (.pkl).
6. Intégration dans le moteur F2 via un module Python dédié (`degradation_ml.py`).

Exigences IA-1

- $RMSE \leq 0,3s/tour$ sur le jeu de test (objectif).
- $R^2 \geq 0,90$ sur données de validation.
- Temps d'inférence $\leq 5ms$ par tour (modèle sérialisé).
- Module de fallback activé si le fichier `.pkl` est absent.
- Notebook Jupyter documentant l'entraînement livré comme artefact.

3.6 IA-2 — Clustering de Circuits (Apprentissage Non Supervisé)

Objectif : Regrouper automatiquement les circuits F1 par profil thermique et caractéristiques d'usure, afin de suggérer des stratégies basées sur des courses passées similaires.

3.6.1 Features de clustering

- Température ambiante moyenne historique (°C).
- Température de piste moyenne (°C).
- Nombre de virages lents / rapides (solllicitation thermique).
- Dégradation moyenne observée par type de gomme (données FastF1).
- Fréquence historique de Safety Car.
- Probabilité de pluie historique.

3.6.2 Pipeline

1. Collecte des données sur les 24 circuits du calendrier 2025.
2. Normalisation MinMaxScaler.
3. Détermination du nombre optimal de clusters : méthode du coude (*Elbow*) + silhouette score.

4. Entraînement K-Means ($k \in [3, 6]$).
5. Attribution d'un label sémantique à chaque cluster (ex. : *Haute Usure Thermique*, *Circuit Froid*, *Tout-temps*).
6. Persistance des résultats en base de données (table `CircuitCluster`).

Exigences IA-2

- Silhouette score $\geq 0,55$ sur la partition finale.
- Les labels de cluster sont affichés dans l'interface F1 au moment de la sélection du circuit.
- Le système suggère automatiquement 3 courses passées sur des circuits du même cluster comme référence stratégique.
- Recalcul du clustering exécutable via commande de management Django (`python manage.py run_clustering`).

4. Données Métier

4.1 Matrice des Écuries

Écurie	γ	T_{stand}	δ	Profil	Caractéristique
Red Bull Racing	0,95	2,0 s	1,4	Élite	Arrêts ultra-rapides
McLaren	0,92	2,1 s	1,3	Optimal	Préservation maximale
Ferrari	1,08	2,3 s	1,7	Agressif	Surchauffe thermique
Haas F1 Team	1,15	2,6 s	1,8	Dégradant	Forte usure sur tendres
Mercedes	1,00	2,4 s	1,5	Neutre	Comportement médian
Aston Martin	1,05	2,5 s	1,6	Variable	Dégrade par haute temp.

TABLE 3 – Matrice des coefficients par écurie

4.2 Tableau des Pneumatiques

5. Architecture Technique

5.1 Stack Technologique

Type	α	Durée max	Conditions	Pénalité hors-cond.	Usage
Soft (S)	0,09	18–22 t	Sec chaud	+7 s/tour si pluie	Quali / relais court
Medium (M)	0,05	26–32 t	Sec neutre	+6 s/tour si pluie	Relais principal
Hard (H)	0,025	35–40 t	Sec froid	+5 s/tour si pluie	Long relais final
Inter (I)	0,06	20–28 t	Piste humide	+4 s/tour si sec	Transition météo
Wet (W)	0,03	25–35 t	Pluie forte	+8 s/tour si sec	Aquaplaning

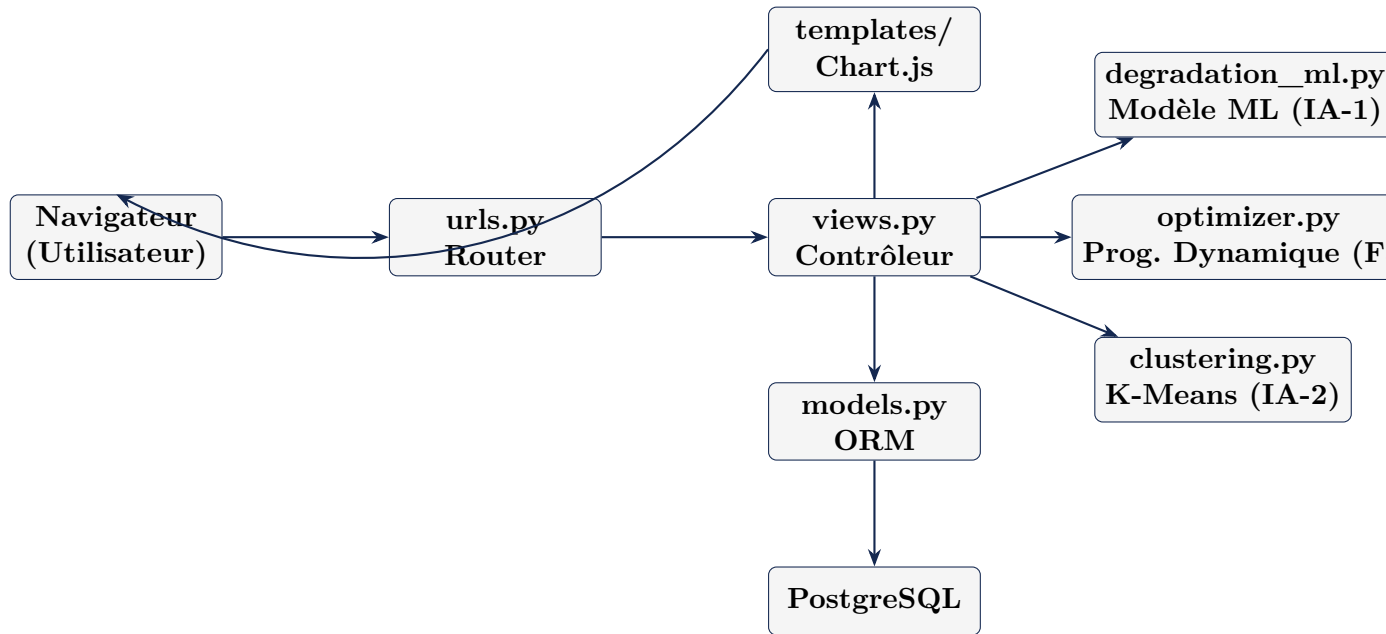
TABLE 4 – Caractéristiques des cinq composés de pneumatiques

Couche	Technologie	Version	Rôle
Langage	Python	3.11+	Logique métier, ML, optimisation
Framework web	Django	4.2+	MVT, ORM, routage HTTP
ML supervisé	scikit-learn	1.4+	Random Forest, XGBoost
Gradient Boost	XGBoost	2.0+	Modèle de dégradation
Calcul	NumPy / Pandas	1.26+	Matrices, features, prog. dynamique
Clustering	scikit-learn	1.4+	K-Means, silhouette
Collecte data	FastF1	3.3+	Télémétrie réelle F1
BDD dev	SQLite	3.x	Prototypage local
BDD prod	PostgreSQL	15+	Production robuste
Visualisation	Chart.js	4.x	Graphiques interactifs
Frontend	HTML5 / CSS3	—	Interface responsive
Déploiement	Docker Compose	24+	Conteneurisation
CI/CD	GitHub Actions	—	Tests automatisés

TABLE 5 – Stack technologique complète de PitSim-F1 v3.0

5.2 Architecture MVT Django

Le projet respecte le pattern MVT de Django, étendu par des modules IA indépendants :



5.3 Organisation du Dépôt

```

pitsim_f1/
    core/                # Configuration Django
    race/                # App principale (F1, F2, F4)
)
    models.py           # Ecurie, Circuit,
CircuitCluster
    views.py
    templates/
    optimizer/          # Module F3
Programmation Dynamique
    dynamic_prog.py     # Algorithme  $O(K*N^2)$ 
    crossover.py        # Detection point de
croisement
    ml/                 # Module IA-1
    degradation_ml.py   # Inference + fallback
    train_model.py      # Script entraînement
    models/degradation.pkl
    clustering/         # Module IA-2
    circuit_clustering.py
    management/commands/run_clustering.py
    static/             # CSS, JS, Chart.js
requirements.txt
docker-compose.yml
Dockerfile
  
```

6. Exigences Non-Fonctionnelles

Critère	Exigence	Méthode de vérification
Performance web	Calcul d'une stratégie complète (70 tours) en < 500 ms côté serveur	Benchmark <code>locust</code>
Inférence ML	Prédiction par tour ≤ 5 ms	Profiling Python
Précision ML	$RMSE \leq 0,3s$, $R^2 \geq 0,90$	Rapport d'entraînement
Couverture tests	$\geq 80\%$ (pytest-django)	Badge CI GitHub
Ergonomie	Interface responsive (mobile, tablette, desktop)	Tests visuels
Déploiement	Une commande <code>docker-compose up</code>	Test d'intégration
Sécurité	Protection CSRF Django ; secrets en variables d'environnement	Revue de code
Extensibilité	Ajout d'une écurie ou d'un circuit sans modifier le code	Admin Django

TABLE 6 – Exigences non-fonctionnelles

7. Planning et Livrables

7.1 Planning Prévisionnel — 14 Semaines

7.2 Livrables Attendus

- L1.** Code source Django complet, versionné sur Git (branche `main` protégée).
- L2.** Dataset F1 nettoyé et annoté (fichier Parquet) avec notebook d'exploration.
- L3.** Modèle ML sérialisé (`degradation.pkl`) + notebook d'entraînement avec rapport de métriques.
- L4.** Résultats du clustering (`circuit_clusters.json`) + rapport de silhouette.
- L5.** Base de données initialisée : 10 écuries 2025, 24 circuits, 5 profils de gommes.
- L6.** Documentation technique : README, docstrings, diagramme entité-relation.
- L7.** Suite de tests automatisés (pytest-django) avec rapport de couverture $\geq 80\%$.
- L8.** Image Docker déployable en production avec `docker-compose`.
- L9.** Rapport de recette avec captures d'écran et résultats de validation.

7.3 Critères d'Acceptation

Phase	Tâche	Semaines	Livrable
1	Modèles ORM, fixtures écuries, circuits	S1–S2	DB initialisée
2	Collecte et nettoyage données FastF1	S2–S4	Dataset nettoyé
3	Entraînement modèle ML (IA-1)	S3–S5	<code>degradation.pkl</code>
4	Moteur de simulation F2 (ML intégré)	S4–S6	Module F2 testé
5	Pipeline clustering circuits (IA-2)	S5–S7	Clusters validés
6	Algorithme de programmation dynamique (F3)	S6–S8	Module F3 testé
7	Module F1 — Interface configuration	S7–S9	UI fonctionnelle
8	Dashboard F4 — Chart.js, exports	S9–S11	Dashboard complet
9	Tests unitaires et d'intégration	S10–S13	Rapport pytest
10	Conteneurisation Docker	S12–S13	Image Docker
11	Recette, documentation, rapport final	S13–S14	Tous livrables

TABLE 7 – Planning prévisionnel sur 14 semaines

8. Glossaire

Stint	Période de course entre deux arrêts aux stands.
Crossover	Tour auquel changer de gomme devient plus rentable que continuer.
RMSE	Root Mean Square Error — métrique d'évaluation des modèles de régression.
K-Means	Algorithme de clustering partitionnel par minimisation de l'inertie.
Silhouette	Métrique mesurant la qualité d'une partition (intervalle $[-1, 1]$).
FastF1	Bibliothèque Python d'accès aux données télémétriques officielles de Formule 1.
ORM	Object-Relational Mapping — abstraction base de données de Django.

ID	Critère	Priorité	Validation
CA-01	Simulation 70 tours sans erreur	Critique	Test unitaire
CA-02	Stratégie calculée en < 500 ms	Critique	Benchmark
CA-03	Pénalité météo correctement appliquée	Critique	Test scénariste
CA-04	RMSE modèle ML $\leq 0,3$ s	Critique	Rapport ML
CA-05	Silhouette clustering $\geq 0,55$	Haute	Rapport clustering
CA-06	5 types de gommes opérationnels	Haute	Test fonctionnel
CA-07	Graphique Chart.js interactif	Haute	Test navigateur
CA-08	Interface responsive mobile	Moyenne	Test visuel
CA-09	Export CSV / PNG fonctionnel	Faible	Test manuel

TABLE 8 – Critères d'acceptation du projet